

Dominando el DOM en JavaScript

Acceso y modificación de Contenido del DOM

En esta lección aprenderás a acceder y modificar el contenido de texto y HTML de los elementos del DOM utilizando las propiedades

clave: `.nodeName`, `.textContent`, `.innerText` y `.outerText`. También exploraremos las diferencias entre estas propiedades para que puedas usarlas de forma efectiva.

Acceder al Nombre de la Etiqueta: `nodeName`

La propiedad `.nodeName` devuelve el nombre de la etiqueta en mayúsculas. Es de solo lectura.

Ejemplo:

```
htmlCopy code<div class="container"></div>

<script>

  const element = document.querySelector(".container");

  console.log(element.nodeName); // "DIV"

</script>
```

Si el nodo no es un elemento HTML, devuelve valores como `#text` para nodos de texto o `#comment` para comentarios.

Modificar Contenido de Texto con `.textContent`

La propiedad `.textContent` permite obtener o modificar el contenido de texto de un elemento, ignorando cualquier marcado HTML.

Ejemplo:

```
htmlCopy code<p class="message">Mi nombre es <strong>Manz</strong>.</p>

<script>

  const element = document.querySelector(".message");

  console.log(element.textContent); // "Mi nombre es Manz."

  element.textContent = "Hola a todos";

  console.log(element.textContent); // "Hola a todos"
```

</script>

Nota: Ignora cualquier etiqueta HTML dentro del elemento y solo devuelve o establece el texto.

Acceso al Texto Renderizado con .innerText

La propiedad .innerText obtiene o modifica solo el texto visible del elemento, respetando los estilos CSS.

Ejemplo:

htmlCopy code<div class="container">

<p>Hola a todos.</p>

<p>Me llamo Manz. <mark style="display: none">Mensaje oculto</mark></p>

</div>

<script>

const element = document.querySelector(".container");

console.log(element.innerText);

// "Hola a todos.

// Me llamo Manz."

</script>

Diferencia clave: Si hay texto oculto por CSS (display: none), .innerText lo ignorará, pero .textContent no.

Reemplazo del Elemento con .outerText

La propiedad .outerText es similar a .innerText, pero al modificar reemplaza la etiqueta completa.

Ejemplo:

htmlCopy code<section><div class="container"></div></section>

<script>

const element = document.querySelector(".container");

element.outerText = "Hola";

```
//<section><div class="container">Hola</div></section>
```

```
element.outerText = "Adiós";
```

```
//<section>Adiós</section>
```

```
</script>
```

Insertar Contenido HTML

Para insertar contenido HTML, no puedes usar `.textContent` o `.innerText`, ya que estas propiedades tratan el texto literalmente. Utiliza `.innerHTML` para este propósito.

Ejemplo:

```
htmlCopy code<div class="container"></div>
```

```
<script>
```

```
const element = document.querySelector(".container");
```

```
element.innerHTML = "<h1>Hola</h1>";
```

```
// Renderiza un encabezado <h1> con el texto "Hola".
```

```
</script>
```

Nota: Evita insertar contenido dinámico no controlado con `.innerHTML` para prevenir vulnerabilidades de seguridad (XSS).

Práctica:

Utiliza las propiedades para lograr los siguientes objetivos en el DOM:

1. **1.** Cambiar el texto de un párrafo oculto.
2. **2.** Mostrar solo el contenido visible de un contenedor.
3. **3.** Reemplazar una etiqueta `<div>` con un mensaje personalizado.

Código Base:

```
htmlCopy code<div class="container">
```

```
<p class="hidden" style="display: none;">Texto oculto</p>
```

```
<p class="replace">Reemplázame</p>
```

```
</div>
```

```
<script>
```

```
// Cambiar texto de párrafo oculto
```

```
const hiddenElement = document.querySelector(".hidden");
```

```
hiddenElement.textContent = "Texto revelado";
```

```
// Mostrar texto visible
```

```
const container = document.querySelector(".container");
```

```
console.log(container.innerText);
```

```
// Reemplazar etiqueta completa
```

```
const replaceElement = document.querySelector(".replace");
```

```
replaceElement.outerText = "Etiqueta reemplazada";
```

```
</script>
```

Acceder a clases CSS

Utiliza el objeto `.classList` para acceder a las clases de un elemento HTML. Consulta cuántas clases tiene, conviértelas en un array y accede a una clase específica.

HTML:

htmlCopy code<div id="page" class="info data dark" data-number="5"></div>

JavaScript:

javascriptCopy codeconst element = document.querySelector("#page");

// Obtener la lista de clases

```
console.log(element.classList);      // ["info", "data", "dark"]
```

```
console.log(element.classList.value); // "info data dark"
```

```
console.log(element.classList.length); // 3
```

// Convertir a array

```
console.log(Array.from(element.classList)); // ["info", "data", "dark"]
```

```
console.log([...element.classList]);      // ["info", "data", "dark"]
```

// Acceder a una clase específica

```
console.log(element.classList.item(0));   // "info"
```

```
console.log(element.classList.item(1));   // "data"
```

```
console.log(element.classList.item(3));   // null
```

Añadir y eliminar clases CSS

Añade nuevas clases CSS a un elemento utilizando `.classList.add()` y elimínalas con `.classList.remove()`. Observa cómo cambia la lista de clases.

HTML:

htmlCopy code<div id="page" class="info data dark"></div>

JavaScript:

```
javascriptCopy codeconst element = document.querySelector("#page");
```

```
// Añadir clases
```

```
element.classList.add("highlight", "selected");
```

```
console.log(element.classList); // ["info", "data", "dark", "highlight", "selected"]
```

```
// Eliminar clases
```

```
element.classList.remove("highlight", "selected");
```

```
console.log(element.classList); // ["info", "data", "dark"]
```

Comprobar si existen clases CSS

Usa `.classList.contains()` para verificar si un elemento tiene una clase específica.

HTML:

```
htmlCopy code<div id="page" class="info data dark"></div>
```

JavaScript:

```
javascriptCopy codeconst element = document.querySelector("#page");
```

```
console.log(element.classList.contains("info")); // true (existe "info")
```

```
console.log(element.classList.contains("hidden")); // false (no existe "hidden")
```

Alternar clases CSS

Utiliza `.classList.toggle()` para alternar una clase en un elemento. Añade la clase si no existe y elimínala si ya está presente.

HTML:

```
htmlCopy code<div id="page" class="info data dark"></div>
```

JavaScript:

```
javascriptCopy codeconst element = document.querySelector("#page");
```

// Alternar clases

```
console.log(element.classList.toggle("info")); // false (la elimina)
```

```
console.log(element.classList); // ["data", "dark"]
```

```
console.log(element.classList.toggle("info")); // true (la añade de nuevo)
```

```
console.log(element.classList); // ["info", "data", "dark"]
```

Reemplazar una clase CSS

Utiliza `.classList.replace()` para reemplazar una clase existente por otra nueva.

HTML:

```
htmlCopy code<div id="page" class="info data dark"></div>
```

JavaScript:

```
javascriptCopy codeconst element = document.querySelector("#page");
```

// Reemplazar clases

```
console.log(element.classList.replace("dark", "light")); // true (cambio exitoso)
```

```
console.log(element.classList); // ["info", "data", "light"]
```

```
console.log(element.classList.replace("hidden", "error")); // false (no existe "hidden")
```

```
console.log(element.classList); // ["info", "data", "light"]
```